



Proyecto Final

Estudiantes: Kevin Daniel Lizano Vargas

Kael Carranza Carrillo

Keitlyn Morales Aguirre

Universidad Autónoma de Centro América

Curso: Sistemas Operativos I

Profesor: Eduardo González Paniagua

Año: 2026

1. Descripción del diseño del Sistema.

1.1 Descripción general.

MiniOS Simulator es un entorno interactivo diseñado para emular y visualizar conceptualmente los mecanismos internos de un sistema operativo moderno. El sistema ofrece una representación gráfica y dinámica sobre cómo el núcleo (kernel) coordina y administra los recursos críticos de hardware —tales como el procesamiento en la CPU, el espacio en memoria principal (RAM), el almacenamiento secundario (Swap) y los ciclos de entrada/salida (E/S)— frente a las demandas de los procesos en ejecución.

Para garantizar la modularidad y una separación clara de responsabilidades, el sistema fue concebido bajo una arquitectura Cliente-Servidor. Esta estructura independiente por completo la interfaz visual de interacción con el usuario de los motores lógicos y matemáticos que calculan las métricas, ejecutan las políticas de planificación y gestionan la memoria virtual.

1.2 Arquitectura del Sistema.

Nuestro proyecto está organizado principalmente por tres componentes: client, server y shared.

El componente client corresponde a la interfaz gráfica, fue desarrollado utilizando React y TypeScript. Esa parte permitir al usuario visualizar el escritorio de simulador, abrir aplicaciones, consultar los procesos, observar el consumo de recursos y también controlar la ejecución de la simulación.

El componente server corresponde al MiniOS Kernel, que fue desarrollado utilizando Node.js. Este componente contiene la lógica principal de la simulación, incluyendo la planificación de procesos, administración de memoria, gestión de Swap, simulación de entrada/salida y actualización del estado de los procesos.

La comunicación entre cliente y el servidor se realizó mediante solicitudes de HTTP a través de una API. Así de esta manera, las acciones realizadas desde la interfaz son enviadas al kernel, el cual procesa la solicitud y devuelve el estado actualizado del sistema.

1.3 Diseño de la interfaz gráfica.

La interfaz de usuario ha sido estructurada de forma intuitiva, dividiéndose en cuatro módulos funcionales que permiten supervisar integralmente la simulación:

1. **Escritorio Interactivo:** Funciona como el punto de entrada de carga de trabajo. Permite al usuario lanzar múltiples aplicaciones simuladas como Word, Chrome, Spotify, Explorador de Archivos, Calculadora, Terminal, VS Code, Paint, Ajustes,

Editor de Texto, Correo, Juegos, Galería de Fotos y Base de Datos. Al seleccionar cualquier aplicación, el sistema instancia un nuevo proceso con parámetros específicos de demanda de CPU, memoria y E/S.

2. **Monitor de Recursos en Tiempo Real:** Dashboard gráfico que despliega el porcentaje global de uso de la CPU, la ocupación del buffer de RAM y Swap, y el volumen de I/O de disco. Asimismo, indica el algoritmo de planificación activo, la duración del quantum y la cantidad acumulada de ticks (pasos de simulación) transcurridos.
3. **Tabla de Control de Procesos (PCB View):** Panel detallado que expone el estado de cada Proceso en el sistema mediante su Bloque de Control de Proceso (PCB). Muestra atributos como: PID (Process ID), Nombre, Estado actual, Consumo de CPU, Memoria Asignada, Operaciones de Disco, Prioridad asignada y Ubicación física/virtual en memoria (RAM o Swap).
4. **Panel de Control de Simulación:** Permite al usuario dirigir la ejecución de la simulación mediante acciones como iniciar, pausar, avanzar paso a paso (step-by-step), reiniciar el entorno, o ajustar parámetros en caliente (cambio de algoritmo de CPU o Quantum).

1.4 Gestión de procesos y planificación de CPU

El simulador implementa el **modelo de estados de procesos**, guiando a cada entidad a través de su ciclo de vida: Nuevo (New), Listo (Ready), Ejecutando (Running), Bloqueado (Blocked), Suspendido (Suspended) y Terminado (Terminated). Cada proceso guarda en su PCB sus requerimientos de cálculo, patrones de lectura/escritura en disco y su tiempo total requerido de ejecución.

El motor de planificación del motor (Kernel Scheduler) abstrae la CPU y permite seleccionar entre cuatro políticas de planificación fundamentales:

- **Round Robin (RR):** Planificación apropiativa (preemptive) donde el usuario puede definir el valor del Quantum. Cada proceso recibe una rebanada de tiempo de CPU fija antes de ser desalojado a la cola de listos.
- **First-Come, First-Served (FCFS):** Planificación no apropiativa atendida en estricto orden de llegada.
- **Shortest Job First (SJF):** Prioriza la ejecución de los procesos con la ráfaga de CPU (burst) estimada más corta.
- **Planificación por Prioridades:** Asigna la CPU en función del nivel de importancia asignado a cada proceso o aplicación.

La progresión temporal se coordina mediante un reloj discreto denominado **tick**. En cada tick, el kernel reevalúa los estados de los procesos, decrementa los tiempos de CPU restantes, procesa las interrupciones de E/S y efectúa los cambios de contexto (context switches) necesarios.

1.5 Gestión de memoria y recursos

El motor de memoria emula una jerarquía de almacenamiento compuesta por una **Memoria RAM de 1536 MB** (memoria principal) y una **Memoria Swap de 8192 MB** (memoria secundaria en disco). Cada proceso requiere una cuota determinada de megabytes para su inicialización y ejecución.

Cuando el consumo de RAM alcanza su límite de capacidad y entra un nuevo proceso o un proceso bloqueado requiere reanudar su ejecución, el administrador de memoria ejecuta las políticas de gestión virtual:

- **Swap Out:** Mueve páginas o bloques de memoria de procesos inactivos o de baja prioridad desde la RAM hacia la memoria Swap en disco, liberando espacio físico.
- **Swap In:** Recupera las páginas o procesos desde el Swap de vuelta a la RAM cuando requieren tiempo de procesamiento activo.
- **Fallos de Página (Page Faults):** El simulador detecta, contabiliza y visualiza las eventualidades en las que un proceso intenta acceder a memoria que no está mapeada en la RAM, disparando la rutina de intercambio requerida y añadiendo la correspondiente penalización por E/S.

1.6 Comunicación entre cliente y servidor

La sincronización entre la interfaz de usuario y el servidor se lleva a cabo mediante una **API RESTful orientada a JSON sobre HTTP**. El cliente actúa de forma declarativa, enviando intenciones del usuario al backend y sustituyendo su estado local con el estado global consolidado que retorna el servidor.

Entre los endpoints y operaciones soportadas por el contrato de la API se destacan:

- GET /api/state: Obtención de la telemetría completa y el snapshot actual de la simulación (procesos, métricas, recursos).
- POST /api/apps/open: Solicitud para instanciar y asignar recursos a un nuevo proceso derivado de una aplicación.
- POST /api/simulation/start | pause | step | reset: Control del flujo de ejecución del reloj de la simulación (ticks).
- PUT /api/config/scheduler: Configuración en tiempo real del algoritmo de planificación de CPU activo y el tamaño del Quantum.

- DELETE /api/processes/:id: Eliminación o cancelación forzada de un proceso específico.
- POST /api/processes/clean: Purga de la tabla de procesos terminados (Terminated) para liberar estructuras de datos en el servidor.

2. Manual de Usuario

Requisitos previos

- Node.js 18 o superior instalado
- Gestor de paquetes npm (incluido con Node.js)

Descarga del proyecto desde GitHub

El código fuente se encuentra en el siguiente repositorio:

<https://github.com/KevinhoProgra/ProyectoFinal>

El proyecto se puede descargar de 2 formas (Clonar con Git o descargar como ZIP) se recomienda la opción de clonar.

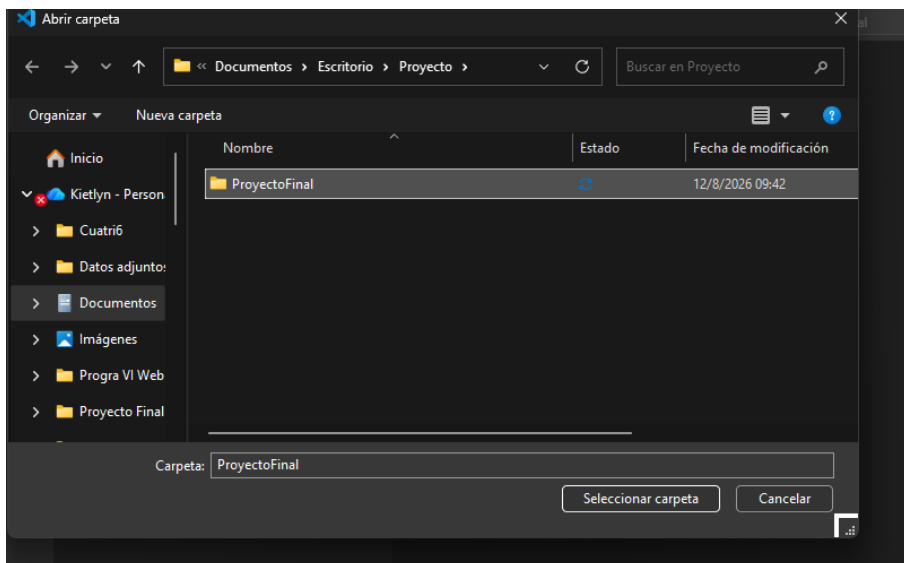
Clonar:

- 1- Abrir una terminal en la carpeta donde se desea guardar el proyecto:
- 2- Ejecutar: “git clone <https://github.com/KevinhoProgra/ProyectoFinal.git>”

Archivo ZIP:

- 1- Ingresar al enlace del repositorio.
- 2- Hacer clic en “Code” y descargar el ZIP.
- 3- Extraer el ZIP en la carpeta donde se desea guardar el proyecto.

Luego de clonarlo o extraer el zip, abrimos el Visual Studio code y abrimos la carpeta “ProyectoFinal” dentro de donde se clono el proyecto.



Instalación y ejecución

El sistema requiere levantar 2 procesos: El cliente (Interfaz) y el servidor (Kernel)

Paso 1: Abrimos una terminal nueva y, en la carpeta de client, ejecutamos el comando “npm install” para instalar las dependencias del proyecto. (solo se utiliza la primera vez)

```
PS C:\Users\keitl\OneDrive\Sistemas Operativos\ProyectoFinal> cd ProyectoFinal
PS C:\Users\keitl\OneDrive\Sistemas Operativos\ProyectoFinal\ProyectoFinal> cd client
PS C:\Users\keitl\OneDrive\Sistemas Operativos\ProyectoFinal\ProyectoFinal\client> npm install
```

En la misma carpeta ejecutamos “npm run dev” para iniciar y obtener la dirección para abrir el simulador. (Abrir en el navegador la URL que indica Vite “ <http://localhost:5173/> “)

```
PS C:\Users\keitl\OneDrive\Sistemas Operativos\ProyectoFinal\ProyectoFinal\client> npm install
PS C:\Users\keitl\OneDrive\Sistemas Operativos\ProyectoFinal\ProyectoFinal\client> npm run dev

> client@0.0.0 dev
> vite

VITE v8.1.5 ready in 4891 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

Nota: La URL puede variar dependiendo de si el puerto se está utilizando o no, pero por defecto es “5173”

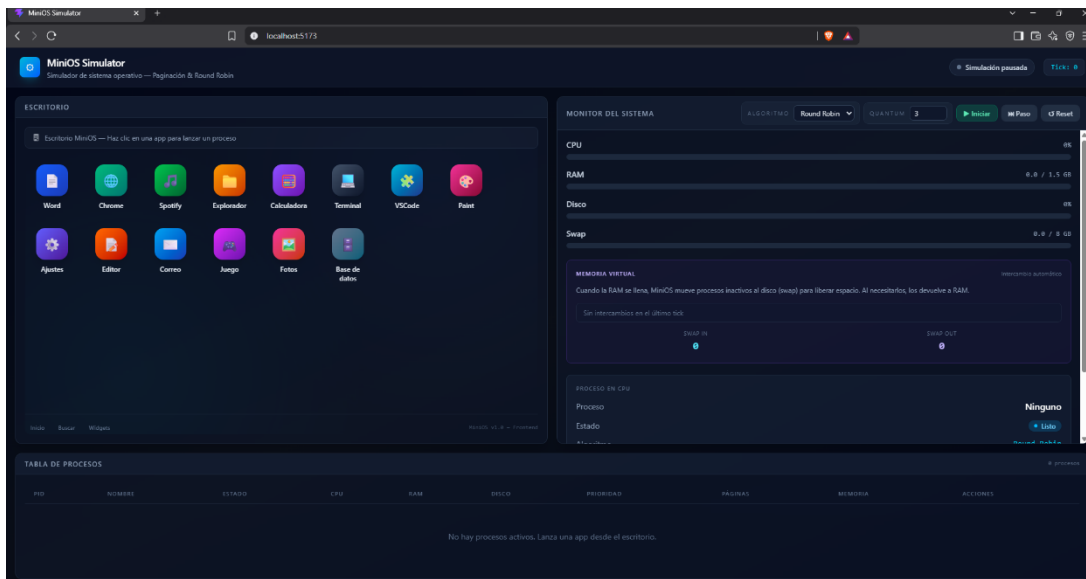
Paso 2: Abrimos una terminal nueva y, en la carpeta de server, ejecutamos “npm start” para iniciar el servidor.

```
PS C:\Users\keitl\OneDrive\Documentos\Desktop\Proyecto\ProyectoFinal> cd server
PS C:\Users\keitl\OneDrive\Documentos\Desktop\Proyecto\ProyectoFinal\server> npm start

> resource-management-simulation-server@1.0.0 start
> node server.js

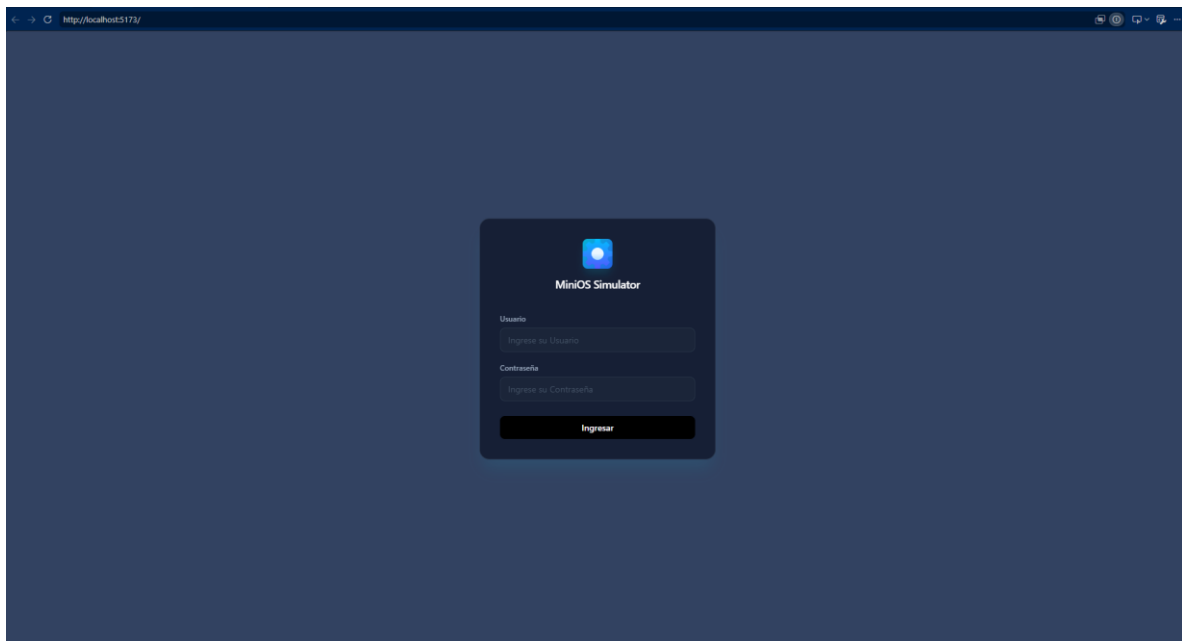
MiniOS Kernel running on http://127.0.0.1:3000
```

Una vez ejecutado los 2 pasos, se accede al navegador web, nos muestra la pantalla principal donde están las aplicaciones, el monitor del sistema, los controles de simulación y la tabla de procesos.

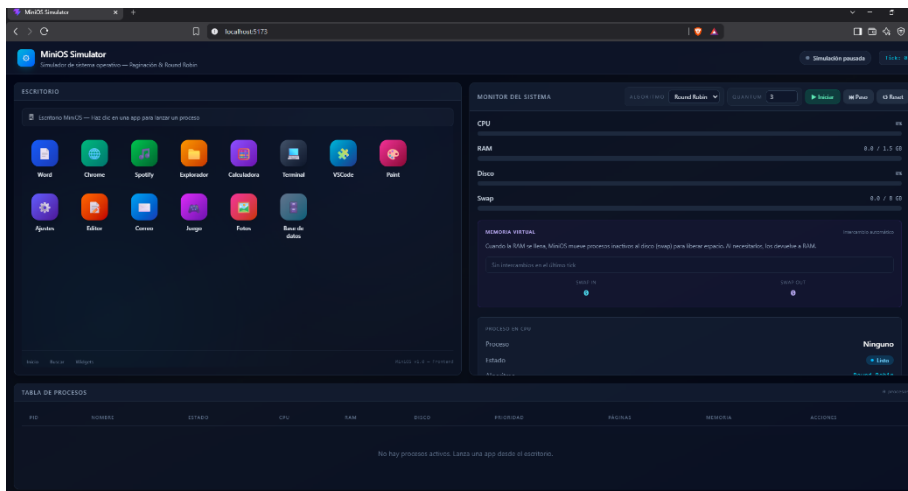


2.1 Inicio del Simulador

Al entrar a la dirección iremos directo a un login, en el cual están los usuarios admin(Password:12345), estudiante (Password:1111), y profesor (Password: abcd), desde un archivo json.

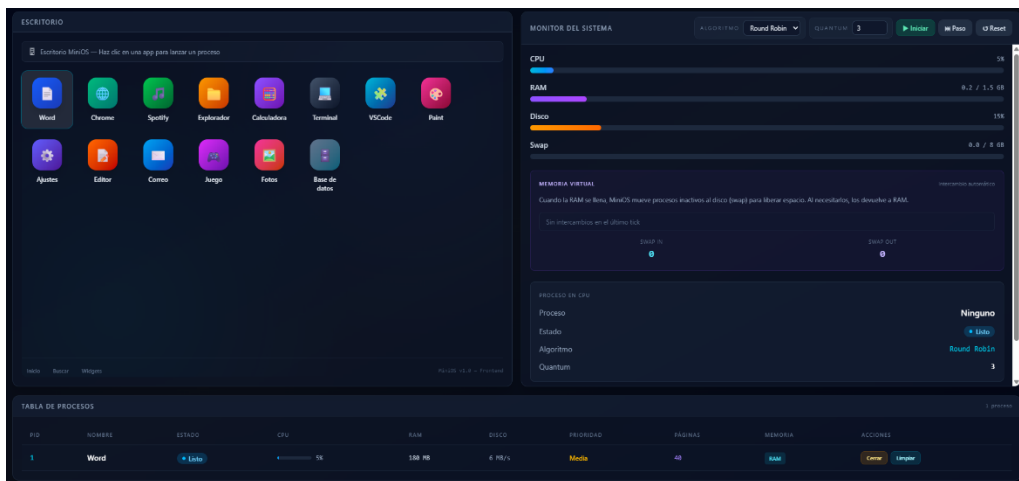


Una vez iniciada la sesión en el proyecto, se accede al navegador web, nos muestra la pantalla principal donde están las aplicaciones, el monitor del sistema, los controles de simulación y la tabla de procesos.



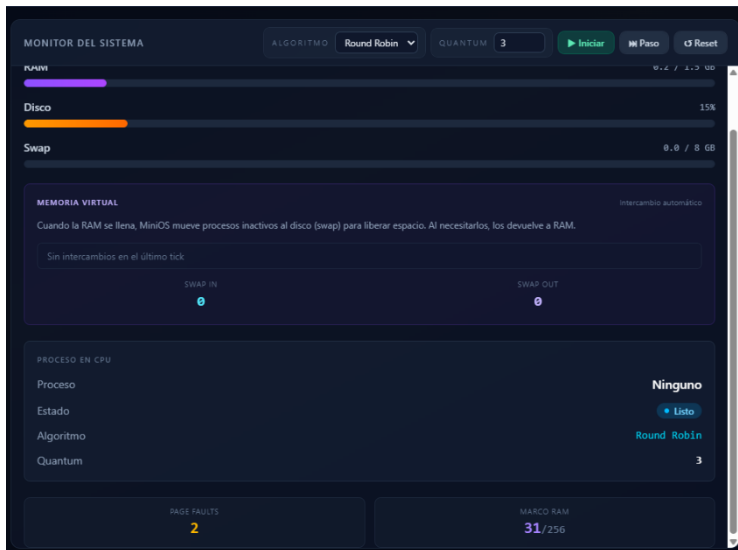
2.2 Apertura de aplicaciones

Seleccionamos una aplicación desde el escritorio y automáticamente se genera un proceso que aparece en la tabla de procesos, cada uno con mismos atributos, pero diferentes valores, dependiendo así de cuanto uso lleva el simulador acorde a la aplicación abierta.



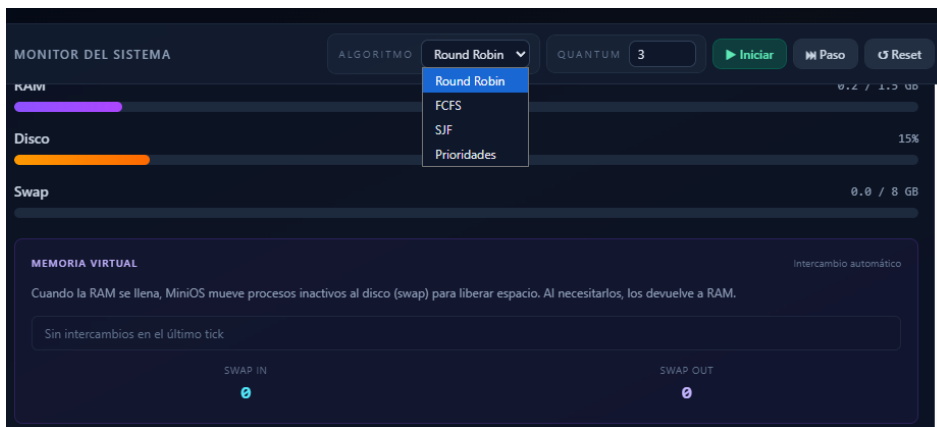
2.3 Control de la simulación

Los controles permiten iniciar, pausar, avanzar paso a paso y reiniciar la simulación, además de la información de Swap y cual aplicación está en swap, el proceso en CPU, su estado, algoritmo y quantum. Por último se pueden apreciar los page faults y marco de RAM.



2.4 Selección del algoritmo

Se puede seleccionar un algoritmo de planificación de CPU entre Round Robin, FCFS, SJF y Prioridades (Arriba en la documentación se puede encontrar como varia el funcionamiento de cada opción).



2.5 Configuración del Quantum

En el algoritmo de planificación **Round Robin (RR)**, el Quantum (o rebanada de tiempo, time slice) representa la cantidad máxima de tiempo continuo de CPU que se asigna a un proceso antes de ser desalojado involuntariamente y devuelto a la cola de listos (Ready Queue).

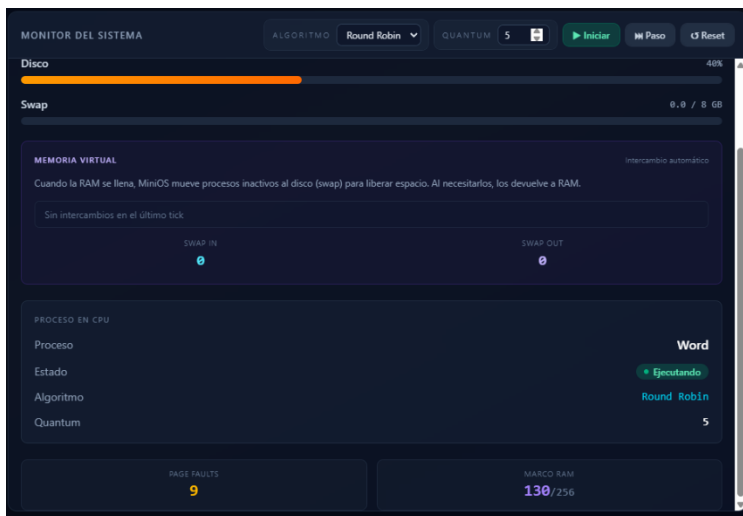
El simulador permite la **configuración dinámica y en caliente** del valor del Quantum mediante la interfaz gráfica, permitiendo al usuario experimentar y analizar directamente el impacto de este parámetro sobre el rendimiento global del sistema:

- **Impacto de un Quantum Pequeño:**

- Ventaja: Mejora drásticamente el tiempo de respuesta (response time) para los procesos interactivos y de corta duración, brindando la ilusión de una ejecución multitarea fluida.
- Desventaja: Incrementa exponencialmente el número de **cambios de contexto** (context switches). Esto genera una mayor sobrecarga (overhead) en el Kernel simulado, dedicando más recursos a la administración de procesos que al procesamiento efectivo de cómputo.

- **Impacto de un Quantum Grande:**

- Ventaja: Reduce la frecuencia de cambios de contexto y la sobrecarga del sistema, maximizando la eficiencia bruta de la CPU para rafagas de cómputo extensas.
- Desventaja: El algoritmo se degrada progresivamente hasta comportarse como un esquema **FCFS (First-Come, First-Served)**, aumentando el tiempo de espera de los procesos más pequeños y pudiendo provocar el efecto convoy.



2.6 Visualización de procesos

La Tabla de Control de Procesos funciona como una representación visual e interactiva del Process Control Block (PCB) del sistema operativo. Su objetivo es brindar una trazabilidad completa e individualizada de cada tarea activa, suspendida o finalizada en la simulación.

A través de esta estructura, el usuario puede inspeccionar métricas y atributos clave en tiempo real:

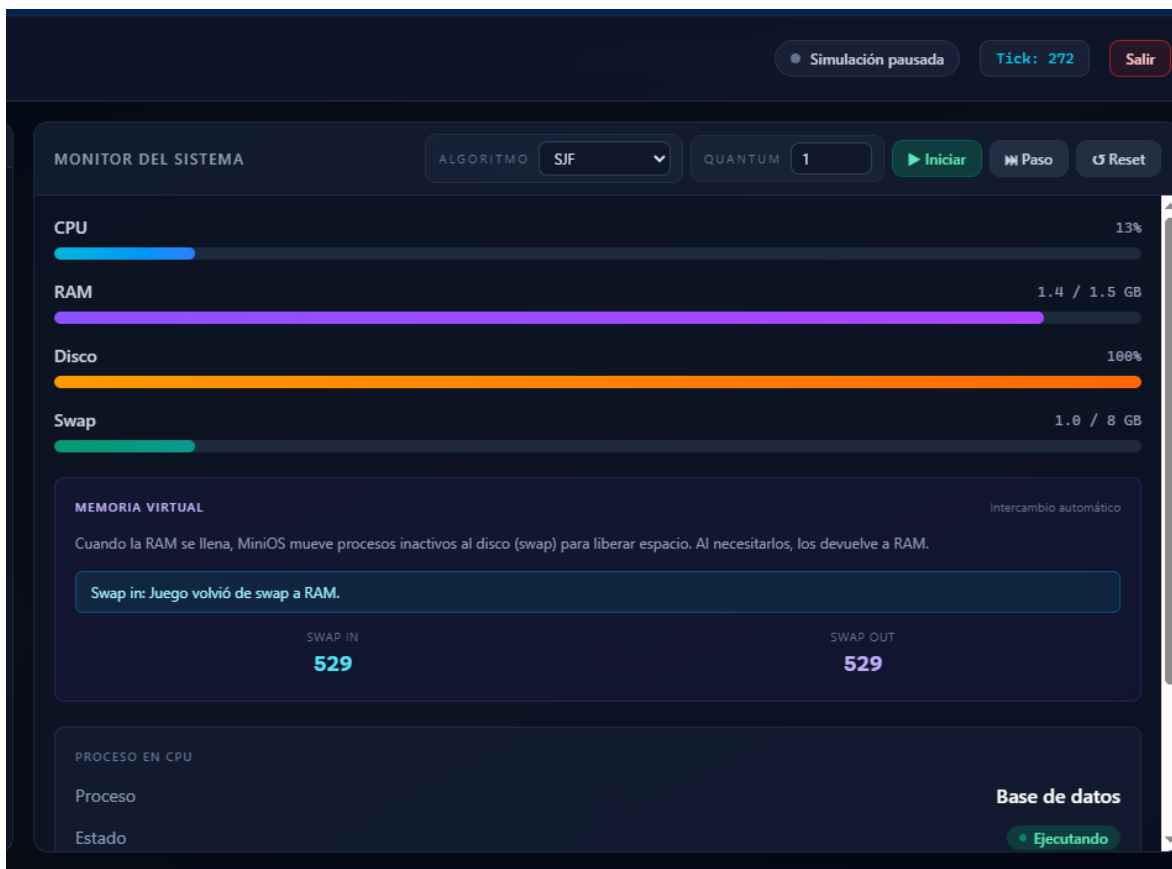
- **Identificación y Atributos Básicos:**
 - **PID (Process ID):** Identificador único asignado secuencialmente por el Kernel al instanciar la aplicación.
 - **Nombre de la Aplicación:** Identificación de la tarea ejecutada (e.g., Chrome, Word, Spotify).
 - **Estado del Proceso:** Indicación precisa de la fase del ciclo de vida en la que se encuentra (New, Ready, Running, Blocked, Suspended o Terminated).
 - **Prioridad:** Nivel jerárquico asignado para la resolución de conflictos en algoritmos de planificación por prioridades.
- **Consumo y Telemetría Estadística de Recursos:** La tabla desglosa el impacto cuantitativo que cada aplicación genera sobre el hardware simulado, facilitando el análisis comparativo del perfil de consumo de los diferentes programas:
 - **Uso de CPU (% / Ráfagas):** Proporción del tiempo de procesamiento consumido y ciclos restantes (burst time) para completar su ejecución.
 - **Asignación de Memoria RAM (MB):** Espacio de memoria principal reservado actualmente por la tarea.
 - **Estructura de Páginas:** Cantidad de marcos de página (page frames) asignados al espacio de direcciones del proceso.
 - **Operaciones de Disco (E/S):** Volumen acumulado de lecturas y escrituras generadas por el proceso hacia el almacenamiento secundario.
- **Indicador de Estado de Paginación y Residencia (In-Memory vs. Swap):** Una de las características más críticas de la tabla es la diferenciación explícita sobre dónde se encuentran físicamente alojados los datos del proceso en cada instante:
 - **Flag de Ubicación de Memoria:** Identifica de forma clara e inequívoca si el espacio de direcciones de la aplicación reside en la Memoria RAM o si ha sido desplazado a la Memoria Swap (Swapped Out).
 - **Trazabilidad del Swapping:** Permite visualizar cuándo un proceso ha sido suspendido o desalojado de la memoria principal debido a la saturación de la RAM, y registrar la ocurrencia de Page Faults cuando el procesador requiere traerlo de vuelta (Swap In) para continuar su ejecución.

Además, de permitir cerrar procesos y limpiar estos mismos de la vista.

PID	NOMBRE	ESTADO	CPU	RAM	DISCO	PRIORIDAD	PÁGINAS	MEMORIA	ACCIONES
1	Word	Ejecutando	15%	180 MB	5 MB/s	Media	40	RAM	Cancelar Limpiar
2	Chrome	En lista	35%	600 MB	10 MB/s	Alta	150	RAM	Cancelar Limpiar
3	Juego	En lista	5%	720 MB	12 MB/s	Alta	180	RAM	Cancelar Limpiar
4	Spotify	En swap	0%	120 MB	2 MB/s	Baja	24	DISCO / SWAP	Cancelar Limpiar

2.9 Datos de la simulación.

Ubicado en el parte superior justo por encima monitor de sistema, se puede apreciar el estado de la simulación (pausada o activa), además de la opción de salir para volver al login y unirse con otro usuario. Cabe recalcar que al simular ser el mismo computador los procesos serán independientemente del usuario ingresado los mismo hasta que estos se finalicen.



3. Documento de análisis que detalle el consumo de recursos y las optimizaciones realizadas.

3.1 Consumo de recursos.

El simulador implementa un modelo integral de monitorización y administración que permite estructurar, cuantificar y visualizar el comportamiento de los recursos críticos del sistema operativo en tiempo real durante la ejecución de los procesos.

- **Gestión y Planificación de CPU:** La unidad central de procesamiento se administra mediante algoritmos de planificación deterministas y adaptativos (como FCFS, SJF, Round Robin o Prioridades). El sistema calcula métricas clave como el tiempo de

espera, el tiempo de retorno (turnaround time), el tiempo de respuesta y la tasa de ocupación de la CPU, permitiendo analizar cómo la ráfaga de CPU (CPU burst) de cada proceso impacta en el rendimiento global.

- **Gestión de Memoria Principal (RAM) y Secundaria (Swap):** La memoria se segmenta dinámicamente para dar cabida a los espacios de dirección de los procesos activos. Cuando la demanda de memoria supera la capacidad física de la RAM, el simulador activa el mecanismo de memoria virtual, delegando el excedente al espacio de Swap (disco).
- **Métricas de Telemetría e Intercambio:** Para representar de manera fiel el comportamiento de un sistema operativo real, se registran y contabilizan los siguientes eventos de E/S y paginación:
 - **Frecuencia de Fallos de Página (Page Faults):** Interrupciones generadas cuando un proceso intenta acceder a una página que no se encuentra cargada en la RAM.
 - **Paginación de Entrada (Swap In):** Operación de lectura donde una página o bloque de memoria se traslada desde la memoria secundaria (Swap) hacia la memoria RAM.
 - **Paginación de Salida (Swap Out):** Operación de escritura donde páginas inactivas o de menor prioridad son desalojadas de la RAM y guardadas en el disco para liberar espacio físico.
 - **Operaciones de Disco (E/S):** Registro acumulativo de lectura y escritura generado por los procesos durante los ciclos de intercambio y gestión de memoria.

3.2 Optimizaciones realizadas.

Con el fin de maximizar la eficiencia en la ejecución y prevenir situaciones de colapso por agotamiento de memoria o starvation de CPU, el simulador incorpora mecanismos estratégicos de optimización:

- **Adaptabilidad Algorítmica en la CPU:** La posibilidad de alternar entre múltiples algoritmos de planificación permite mitigar problemas clásicos como el efecto convoy o la inanición, optimizando el throughput (rendimiento) del sistema según la carga de trabajo simulada.
- **Estrategias de Reemplazo de Paginación y Memoria Virtual:** La integración de un esquema de memoria Swap previene que el sistema colapse por falta de memoria física. Mediante políticas de reemplazo (como FIFO, LRU o Segunda Oportunidad),

el simulador identifica qué páginas deben ser trasladadas al almacenamiento secundario (Swap Out) de manera eficiente, minimizando la sobrecarga (overhead) producida por la hiperpaginación (thrashing).

- **Liberación Dinámica y Gestión de Estados:** El simulador realiza una recolección continua de recursos asignados a procesos que alcanzan el estado de finalizado (Terminated). Al eliminar las estructuras de control de proceso (PCB) y liberar los marcos de página (page frames) en RAM y bloques en Swap, se garantiza que la memoria esté inmediatamente disponible para nuevos procesos.
- **Desalojo Preventivo (Swapping de Procesos Completos):** En escenarios de alta saturación de memoria RAM, el sistema puede suspender temporalmente procesos en estado bloqueado o de baja prioridad, moviendo su estado completo a la memoria secundaria para desbloquear la ejecución de procesos críticos o de alta prioridad en la memoria principal.